

1° Instancia Evaluativa

Interfaz Gráfica

Proyecto: Simulador de Ecosistema

Información General

- Examen práctico grupal (3-4 integrantes)
- Entrega: 08 mayo 2026

Entregables obligatorios:

Código del Proyecto (obligatorio)

- Link del proyecto completo
- Pueden utilizar GitHub, en ese caso colocarán el link al repositorio
- En caso de utilizar GitHub:
 - TODOS los integrantes deben tener commits realizados
 - **Penalización:** integrante sin commits en el código pierde el 50% de la nota grupal

Video explicativo (obligatorio)

- Duración: entre 10 y 15 minutos
- Todos los integrantes con cámara encendida
- La explicación de cada integrante puede ser individual y luego unirse en un solo video
- Cada integrante debe mostrar su pantalla con el código al momento de explicar
- Se debe explicar: la funcionalidad implementada, el código técnico, los conflictos encontrados, cómo los resolvieron, y uso transparente de IA u otra documentación
- Se evaluará especialmente la comprensión individual de cada parte del código implementada
- Pueden usar Vimeo para capturar pantalla y rostro. Sin webcam: grabarse con el celular y unir con Canva

Documentación (obligatorio)

- En la carpeta del proyecto debe haber una carpeta con capturas de los prompts utilizados, o un Word con los links a las conversaciones completas con la IA, o links de páginas web consultadas
- Esta entrega es **INDIVIDUAL**. Deberán dejar claro de quién es cada captura
- Alternativa: agregar los links de las conversaciones completas en un README.md

README.md (opcional, recomendado)

- Instrucciones de ejecución (NetBeans version, librerías, etc.)

- Estructura del proyecto
- Integrantes y rol de cada uno
- Desafíos encontrados
- Uso de IA / herramientas externas

CRÍTICO: La falta de cualquiera de los 3 entregables obligatorios anula la instancia evaluativa del alumno

Nota: esta es la parte práctica de la evaluación. El día de la entrega, cada integrante tendrá que realizar un cuestionario individual y entregar el link del repositorio.

Objetivo del Proyecto

Desarrollar en Java una simulación de ecosistema por turnos que corre en la terminal. El jugador configura el ecosistema inicial y puede intervenir cada ciertos turnos tomando decisiones que afectan el equilibrio del sistema.

El ecosistema contiene tres tipos de entidades: plantas, conejos y lobos. Cada turno las entidades interactúan entre sí según reglas definidas, consumen o ganan energía, se reproducen o mueren. El jugador observa la evolución y puede intervenir para cambiar el clima o agregar entidades.

Este proyecto aplica de forma integrada: herencia, clases abstractas, interfaces, polimorfismo, encapsulamiento, colecciones y estructuras de control.

Mecánica del juego

Configuración inicial

Al iniciar, el jugador ingresa:

- Cantidad inicial de plantas (mínimo 5, máximo 30)
- Cantidad inicial de conejos (mínimo 2, máximo 15)
- Cantidad inicial de lobos (mínimo 1, máximo 5)
- Clima inicial: Soleado, Lluvioso, Sequía o Invierno
- Cantidad de turnos totales de la simulación (entre 10 y 50)

Cada turno, en orden

- Las plantas con suficiente energía se reproducen
- Los conejos buscan plantas cercanas y comen (si no encuentran, pierden energía)
- Los lobos intentan cazar un conejo (probabilidad según energía del lobo)
- Todas las entidades envejecen y gastan energía base por existir
- Las entidades sin energía mueren
- Se muestra el estado del ecosistema: conteos y eventos del turno
- Cada 3 turnos: el jugador puede intervenir o simplemente avanzar

Clima y sus efectos

Clima	Efecto
Soleado	Plantas: reproducción x1.5 Conejos: energía base +5 Lobos: sin cambio
Lluvioso	Plantas: reproducción x2 Conejos: energía base +3 Lobos: -5 energía por turno
Sequía	Plantas: reproducción x0.5 Conejos: -5 energía por turno Lobos: sin cambio
Invierno	Plantas: no se reproducen Conejos: -8 energía por turno Lobos: caza con +20% éxito

Intervención del jugador (cada 3 turnos)

- Cambiar el clima
- Agregar entidades (planta, conejo o lobo)
- Solo avanzar turno sin intervenir

Condiciones de fin

- Se alcanzó el número de turnos configurado
- Se extinguieron los conejos O los lobos O las plantas (colapso del ecosistema)

Al finalizar se muestra un reporte completo de la simulación.

Clases Obligatorias

Clase abstracta Entidad

Clase base de todas las entidades del ecosistema.

- Atributos: nombre (String), energía (double), edad (int), viva (boolean)
- Constructor con parámetros
- Métodos abstractos: actuar(Ecosistema eco), mostrarEstado()
- Método concreto: envejecer() — incrementa edad y descuenta energía base
- Getters y setters con validación (energía no puede ser negativa: se lleva a 0)

Clase Planta (extends Entidad, implements Reproducible)

- Atributos adicionales: tamaño (int, 1-5) — afecta cuánta energía da al ser comida
- actuar(): intenta reproducirse si tiene suficiente energía y el clima lo permite
- mostrarEstado(): imprime nombre, tamaño y energía
- serComida(): reduce energía al mínimo y retorna el valor nutritivo (tamaño * 10)

Clase abstracta Animal (extends Entidad, implements Mortal)

- Atributos adicionales: velocidad (int), peso (double)
- Método abstracto: comer(Ecosistema eco)
- Método concreto: moverse() — imprime que el animal se desplazó

Clase Conejo (extends Animal, implements Reproducible)

- actuar(): llama a comer() y luego intenta reproducirse
- comer(): busca una planta viva en el ecosistema y la come; si no hay, pierde 15 de energía
- mostrarEstado(): imprime nombre, energía y si está en peligro (energía < 20)
- Reproducción: si energía > 60 y hay al menos otro conejo vivo, puede generar un nuevo Conejo

Clase Lobo (extends Animal)

- Atributos adicionales: exitosCaza (int) — contador de cacerías exitosas
- actuar(): intenta cazar un conejo vivo
- comer(): selecciona un conejo aleatorio; si la caza es exitosa (probabilidad según energía), lo mata y gana energía
- mostrarEstado(): imprime nombre, energía y cantidad de cacerías exitosas
- Nota: Lobo no implementa Reproducible — los lobos no se reproducen en esta simulación

Interface Reproducible

- Métodos: reproducirse(Ecosistema eco), puedeReproducirse()
- Método default: intentarReproduccion(Ecosistema eco) — llama a puedeReproducirse() y si retorna true, llama a reproducirse()
- La implementan: Planta y Conejo
- Se usa con polimorfismo: en el Ecosistema, un ArrayList<Reproducible> permite procesar la reproducción de plantas y conejos en un mismo recorrido

Interface Mortal

- Métodos: estaVivo(), morir()
- Método default: verificarMuerte() — si energía <= 0, llama a morir() e imprime el evento
- La implementan: Conejo y Lobo (y opcionalmente Planta)

Clase Ecosistema

- Atributos: ArrayList<Planta> plantas, ArrayList<Conejo> conejos, ArrayList<Lobo> lobos, Clima climaActual, int turnoActual
- procesarTurno() — ejecuta el orden de acciones de cada turno
- mostrarEstado() — imprime el conteo actual de cada entidad y el clima
- agregarEntidad(String tipo) — agrega una nueva entidad según tipo
- cambiarClima(Clima nuevo) — actualiza el clima

- `ecosistemaColapsado()` — retorna true si alguna población llegó a 0
- `generarReporteFinal()` — imprime el resumen completo al finalizar

Funcionalidades a implementar

1. Configuración inicial

- Menú de configuración con Scanner que valide todos los rangos indicados
- Crear las entidades iniciales con energía aleatoria dentro de rangos razonables
- Confirmar la configuración antes de iniciar

2. Loop principal de simulación

- Implementar con un bucle que avance turno a turno hasta la condición de fin
- El jugador presiona Enter para avanzar cada turno
- Mostrar los eventos del turno de forma legible (quién comió a quién, quién murió, quién nació)
- Mostrar el estado general del ecosistema al final de cada turno

3. Sistema de intervención

- Cada 3 turnos, interrumpir el loop y mostrar el menú de intervención
- Validar que no se puedan agregar más de 5 lobos en total en toda la simulación
- Confirmar la acción elegida antes de ejecutarla

4. Reporte final

- Causa de fin (turnos completos o colapso — especificar cuál población se extinguió)
- Turno de mayor actividad (el turno con más eventos registrados)
- Entidad más longeva de cada tipo (mayor edad al finalizar)
- Lobo con más cacerías exitosas
- Total de nacimientos y muertes por tipo a lo largo de toda la simulación

5. Requisitos de POO (obligatorios para aprobar)

- Entidad abstracta con métodos abstractos correctamente sobreescritos
- Animal como capa intermedia de herencia con método concreto compartido
- Reproducible implementada por Planta y Conejo, usada con polimorfismo
- Mortal implementada por los animales, con método default aprovechado
- Encapsulamiento completo: todos los atributos private con getters/setters validados
- Sobrecarga: al menos un método con 2 versiones (sugerido: `agregarEntidad` con y sin energía inicial)

BONUS (puntos extras)

Nueva entidad: Planta Venenosa (extends Planta)

- Si un conejo la come, pierde 30 de energía en vez de ganar

- El conejo no puede distinguirla de una planta normal antes de comerla
- Demostrar polimorfismo: se almacena en el mismo ArrayList<Planta>

Estadísticas en tiempo real

- Llevar un historial del conteo de cada población turno a turno
- Al finalizar, mostrar en qué turno cada población llegó a su máximo y a su mínimo

Interface Peligroso

- Método: getNivelPeligro() → int
- La implementa Lobo (y opcionalmente PlantaVenenosa)
- En el reporte final, listar los elementos peligrosos del ecosistema ordenados por nivel

BONUS: Los puntos obtenidos en los ítems extras se suman al puntaje final del proyecto para mejorar la nota global de la IE

Ejemplo de salida por turno

Referencia de cómo debería verse la terminal durante la simulación:

```
=== TURNO 7 | Clima: Soleado ===
Plantas: 14   Conejos: 8   Lobos: 3

-- Eventos --
Planta 'Helecho-3' se reprodujo → nueva planta 'Helecho-9' (energia: 40)
Conejo 'Blas' comió 'Helecho-2' (+20 energia)
Conejo 'Luna' no encontró comida (-15 energia) [PELIGRO: energia=12]
Lobo 'Fang' cazó a Conejo 'Topo' (+30 energia) [cacerías: 4]
Lobo 'Sombra' falló la caza
Conejo 'Rex' murió de inanición

Estado: Plantas: 15   Conejos: 7   Lobos: 3

>>> Presione Enter para continuar...
```

```
=== INTERVENCIÓN (cada 3 turnos) ===
1. Cambiar clima   (actual: Soleado)
2. Agregar entidad
3. Solo avanzar
Opción: 2
¿Qué entidad agregar? (planta/conejo/lobo): planta
Se agregó 'Helecho-16' al ecosistema.
```

Distribución de puntaje de la 1° IE

El puntaje se compone de dos partes que deben aprobarse de forma independiente:

- Cuestionario individual: 100 puntos → 40% de la nota global (mínimo para aprobar: 55/100)
- Proyecto grupal: 100 puntos → 60% de la nota global (mínimo para aprobar: 55/100)

Si se desaprueba cualquiera de las dos partes, la IE queda desaprobada con un 3 o menos.

Escala de valoración

Nota	Puntaje
1	01 – 24
2	25 – 39
3	40 – 54
4 (aprueba)	55 – 61
5	62 – 66
6	67 – 72
7 (promociona)	73 – 79
8	80 – 87
9	88 – 96
10	97 – 100